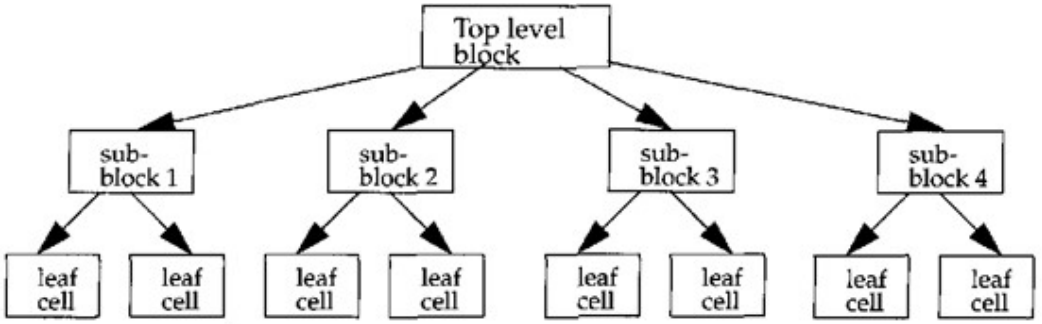
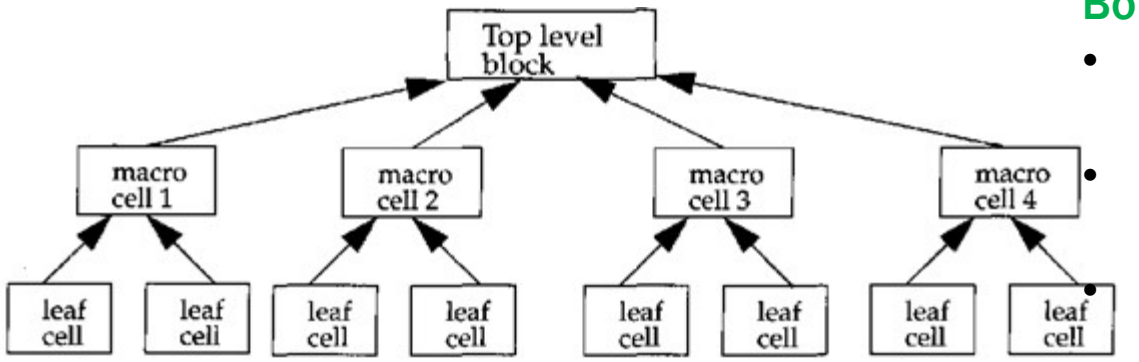
Design Methodologies



Top-Down design methodology

- Define the top level block and identify the sub-blocks necessary to build the top-level block.
- Further sub divide the sub-blocks until come to leaf cells, which are the cells that cannot further be divided.



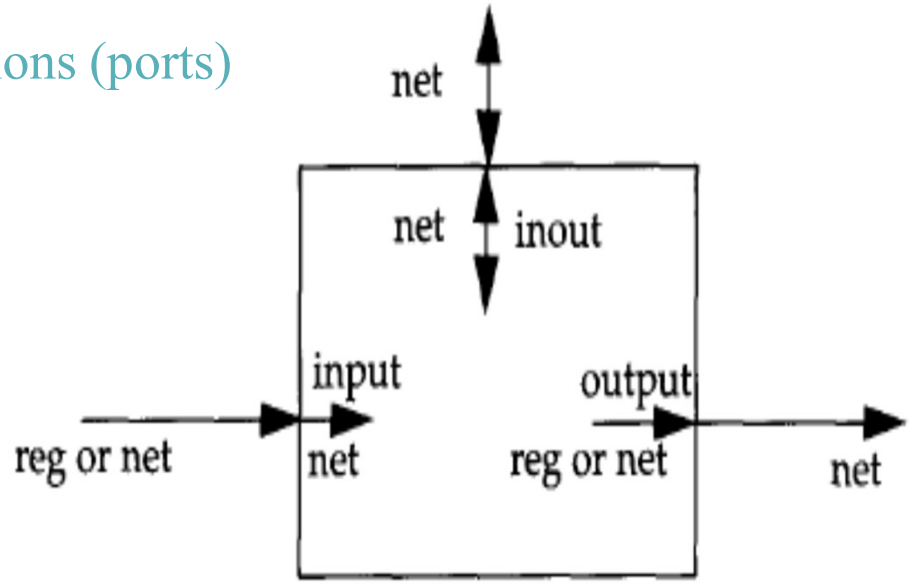
Bottom-Up design methodology

- Identify the building blocks that are available to us.
- Build bigger cells, using these building blocks.
- These cells are then used for higher-level blocks until we build the top-level block in the design.

Hierarchical Modeling with Verilog: Defining a module

- A **module** is the main building block in Verilog
  - Instanced into a design (never called)
- We first need to declare:
  - Name of the module
    - Name can't begin with a number
    - Names are case sensitive
  - Types of its external connections (ports)
    - input, output, inout
  - Names of its connections
  - Functionality of the module

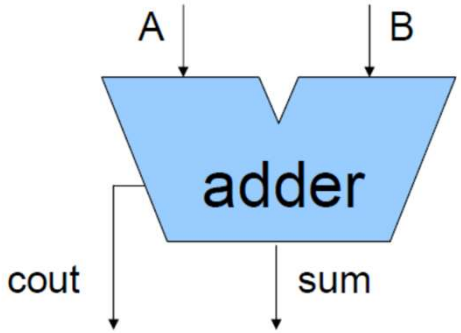
module **<module_name>**
(<module_terminal_list>);
<module_terminal_definitions>
…
<functionality_of_module>
…
endmodule



Hierarchical Modeling with Verilog: port list

- module includes a name and an interface in the form of a port list
  - must specify direction and bitwidth for each port

```
1   module adder( A, B, cout, sum );
2       input [3:0] A, B;
3       output cout;
4       output [3:0] sum;
5
6       // adder functionality
7
8   endmodule
```




```
1   module adder(input [3:0] A, B,
2                output cout,
3                output [3:0] sum );
4
5       // adder functionality
6
7   endmodule
```

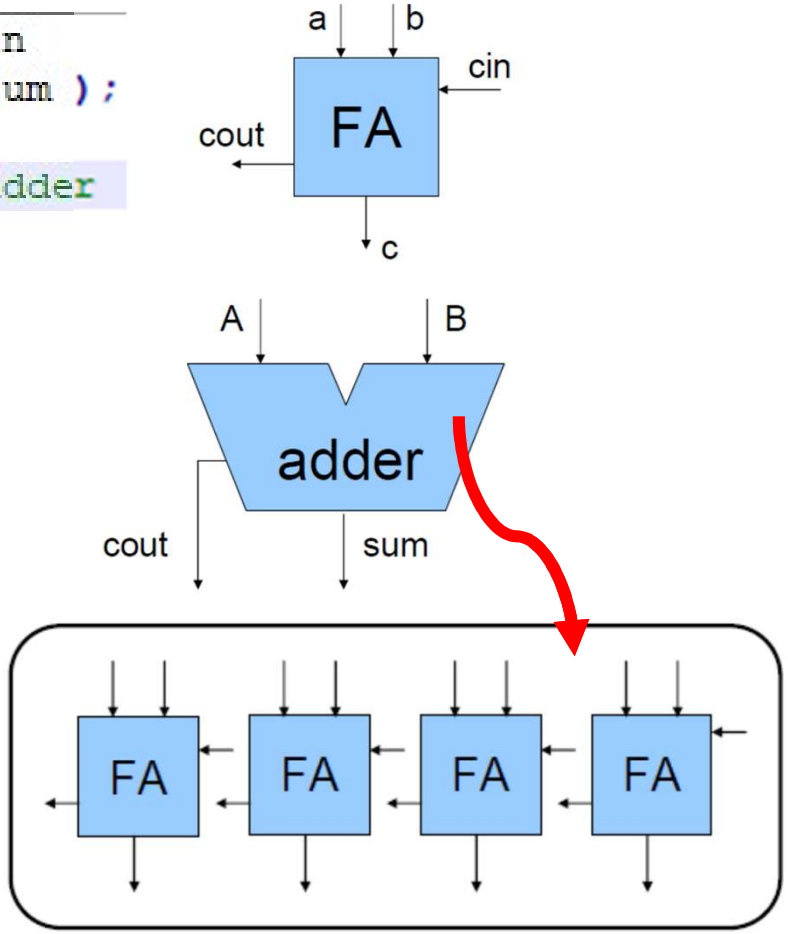
Verilog-2001 introduced an ANSI C style portlist

Hierarchical Modeling with Verilog: instantiation

- module can contain other modules through **module instantiation** creating a module hierarchy

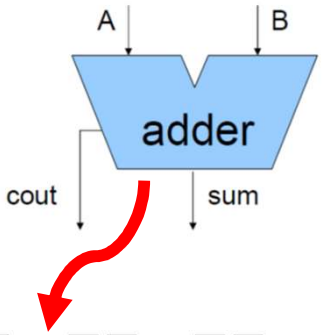
```
1  module FA(  input a, b, cin
2              output cout, sum );
3
4      // modeling of 1 bit adder
5
6  endmodule
```

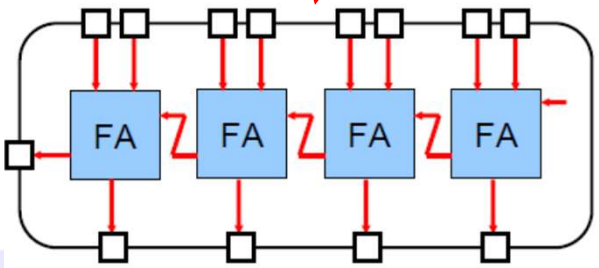
```
1  module adder( input [3:0] A, B,
2                output cout,
3                output [3:0] S );
4                FA fa0( ... );
5                FA fa1( ... );
6                FA fa2( ... );
7                FA fa3( ... );
8  endmodule
```



Hierarchical Modeling with Verilog: instantiation

- Modules are connected together with nets
- Ports are attached to nets either by:





position

```
module adder(
    input [3:0] A, B,
    output cout,
    output [3:0] S );
    FA fa0( A[0], B[0], 0, c0, S[0] );
    FA fa1( A[1], B[1], c0, c1, S[1] );
    FA fa2( A[2], B[2], c1, c2, S[2] );
    FA fa3( A[3], B[3], c2, cout, S[3] );
endmodule
```

```
module FA(  input a, b, cin
            output cout, sum );

    // modeling of 1 bit adder

endmodule
```

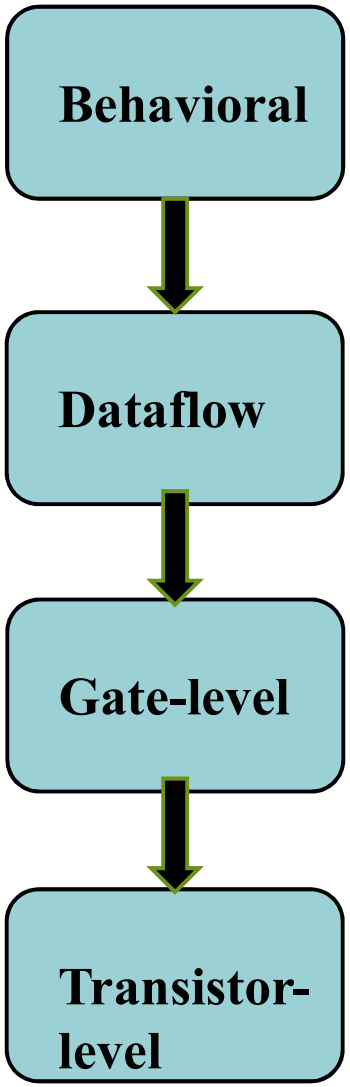
name

```
module adder(
    input [3:0] A, B,
    output cout,
    output [3:0] S );
    FA fa0(.a(A[0]), .b(B[0]),.cin(0), .cout(c0),.sum(S[0] );
    FA fa1(.a(A[1]), .b(B[1]),.cin(c0), .cout(c1),.sum(S[1]),
    FA fa2(.a(A[2]), .b(B[2]),.cin(c1), .cout(c2),.sum(S[2]),
    FA fa3(.a(A[3]), .b(B[3]),.cin(c2), .cout(cout),.sum(S[3]);
endmodule
```

Connecting ports to external signals

- Connecting by ordered list
  - More intuitive for beginners
  - Mostly used when having few ports
- Connecting by name
  - Used when having more ports
  - Gives independence from order of ports

Verilog Abstraction Levels



Module's high-level algorithm is implemented with little concern for the actual hardware

Module is implemented by specifying how data flows between registers

Module is implemented in terms of concrete logic gates (AND, OR, NOT) and their interconnections

Module is implemented in terms of pmos, and nmos transistors and their interconnections

Which one to choose ?

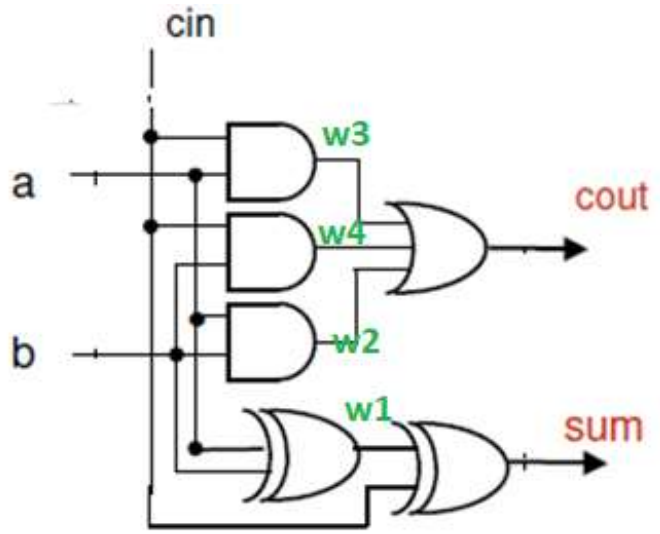
- Designers usually use a **mix of all three**! Early on in the design process they might use mostly behavioral models. As the design is refined, the behavioral models begin to be replaced by dataflow models.
- Finally, the designers use automatic tools to synthesize a low-level gate-level model.

Gate Level Modeling

- Module implemented in terms of logic gates like (and ,or, …) and interconnection between gates
- A logic circuit can be designed by use of logic gates.
- Verilog supports basic logic gates as predefined *primitives.* These primitives are instantiated like modules except that they are predefined in Verilog and do not need a module definition.

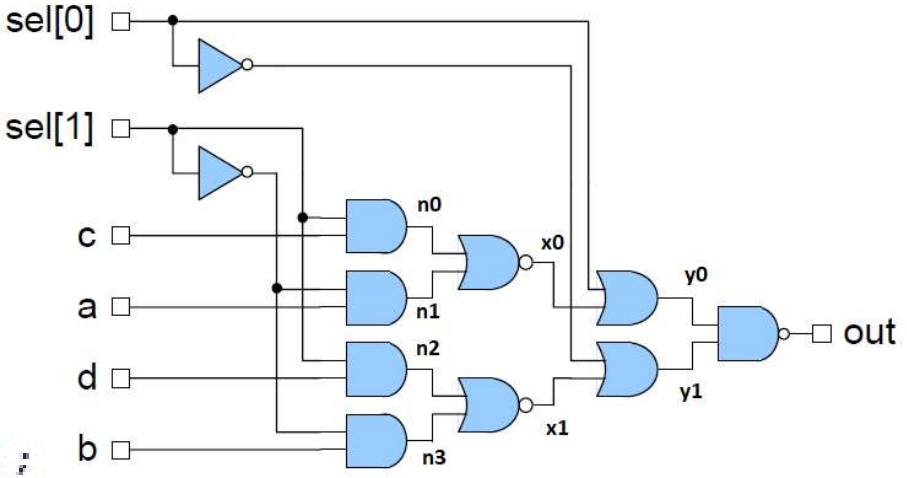
Gate-Level: Full Adder

```
1   module FullAdder(
2         a,b,cin,cout,sum);
3         input a, b, cin;
4         output cout, sum;
5         wire w1, w2, w3, w4;
6         xor (w1, a, b);
7         xor (sum, w1, cin);
8         and (w2, a, b);
9         and (w3, a, cin);
10        and (w4, b, cin);
11        or  (cout, w2, w3, w4);
12  endmodule
```



Basic logic gates are built-in primitives meaning there is no need to define a module for these gates

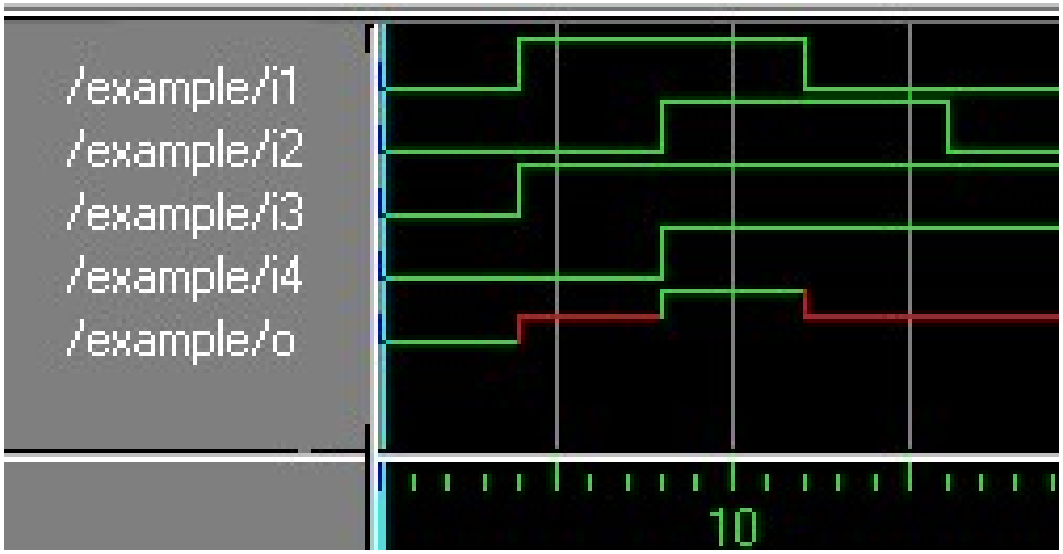
Gate-Level: 4-input Multiplexer

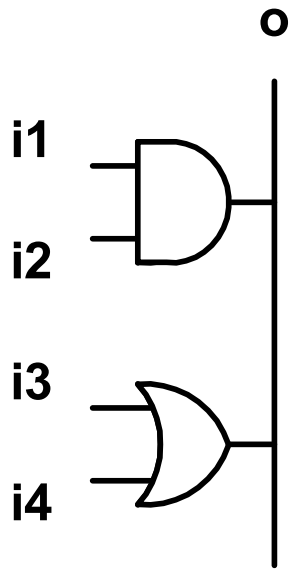



```
1  module mux4( input a, b, c, d
2               input [1:0] sel,
3               output out );
4      wire [1:0] sel_b;
5      not not0( sel_b[0], sel[0] );
6      not not1( sel_b[1], sel[1] );
7      wire n0, n1, n2, n3;
8      and and0( n0, c, sel[1] );
9      and and1( n1, a, sel_b[1] );
10     and and2( n2, d, sel[1] );
11     and and3( n3, b, sel_b[1] );
12     wire x0, x1;
13     nor nor0( x0, n0, n1 );
14     nor nor1( x1, n2, n3 );
15     wire y0, y1;
16     or or0( y0, x0, sel[0] );
17     or or1( y1, x1, sel_b[0] );
18     nand nand0( out, y0, y1 );
19 endmodule
```

Example: a net with two drivers

```
1   module example;
2     reg i1,i2,i3,i4;
3     wire o;
4
5     and g1(o,i1,i2);
6     or  g2(o,i3,i4);
7
8   endmodule
```





User Defined Primitives

```
1   //User defined primitive(UDP)
2   primitive crctp (x,A,B,C);
3   output x;
4   input A,B,C;
5   //Truth table for
6   //x(A,B,C) = Minterms (0,2,4,6,7)
7   table
8   //  A B C : x (only for comment)
9       0 0 0 : 1;
10      0 0 1 : 0;
11      0 1 0 : 1;
12      0 1 1 : 0;
13      1 0 0 : 1;
14      1 0 1 : 0;
15      1 1 0 : 1;
16      1 1 1 : 1;
17  endtable
18  endprimitive
```

Dataflow Level Modeling

- In complex designs the number of gates is very large so module could be designed by specifying dataflow.
- The designer is aware of how data flows between hardware registers and how the data is processed in the design
- Currently, automated tools are used to create a gate-level circuit from a dataflow design description. This process is called ***logic synthesis***

Assignments

- ***Continuous assignments***
  - assigns values to nets
- ***Procedural assignments***
  - assigns values to registers

Continuous Assignments

- Continuous assignments use the **assign** keyword
- Continuous assignments drive values onto **net** (vector and scalar)
- The assignment occurs whenever simulation causes the value of the right-hand side (RHS) to change
- They provide a way to model combinational logic specifying the logical expression that drives the net instead of an interconnection of gates
- Whenever an operand in the right hand side changes during simulation, the whole right-hand side expression is evaluated and assigned to the left-hand side
- Modeling a 16-bit adder with a continuous assignment

```
1    wire [15:0] sum, a, b;  // declaration of 16-bit vector nets
2    wire cin, cout;         // declaration of 1-bit (scalar) nets
3    assign {cout, sum} = a + b + cin;
```

Dataflow : 4-input Multiplexer

```
1  module mux4( input a, b, c, d,
2               input [1:0] sel,
3               output out );
4      wire out, t0, t1;
5      assign t0 = ( (sel[1] & c) | (~sel[1] & a) );
6      assign t1 = ( (sel[1] & d) | (~sel[1] & b) );
7      assign out = ( (t0 & sel[0]) | (t1 & ~sel[0]) );
8  endmodule
```

This is called a continuous assignment since the RHS is always being evaluated and the result is continuously being driven onto the net on the LHS

Dataflow : 4-input Multiplexer

```
1  module mux4( input a, b, c, d,
2               input [1:0] sel,
3               output out );
4    assign out = ( sel == 0 ) ? a :
5                 ( sel == 1 ) ? b :
6                 ( sel == 2 ) ? c :
7                 ( sel == 3 ) ? d : 1'bx;
8  endmodule
```

Verilog Dataflow style enables descriptions which are more

abstract than gate-level Verilog

Dataflow : Key Points

- Dataflow modeling enables the designer to focus on where the state is in the design and how the data flows between these state elements without becoming bogged down in gate-level details
- Continuous assignments are used to connect combinational logic to nets and ports
- A wide variety of operators are available including:

```
Arithmetic:     + - * / % **
Logical:        ! && ||
Relational:     > < >= <=
Equality:       == != === !===
Bitwise:        ~ & | ^ ^~
Reduction:      & ~& | ~| ^ ^~
Shift:          >> << >>> <<<
Concatenation:  { }
Conditional:    ?:
```

Avoid these operators since they usually synthesize poorly

Invalid sequential assigns

```
assign A = X | (Y & ~Z);

assign B = W | A;

assign A = Y & Z;
```

"Reusing" a variable on the left side of several assign statements is not allowed

```
assign A = X | (Y & ~Z);

assign B = W | A;

assign X = B & Z;
```

Cyclic dependencies are also bad

A depends on **X**
which depends on **B**
which depends on **A**

Procedural Assignments

- Procedural assignments drive values onto ***reg*** (vector and scalar)
- The register data type holds the value of the assignment until the next procedural assignment to the register
- They occur within procedures such as ***always*** *and* ***initial***

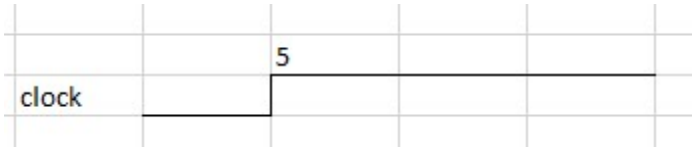
- ***initial***
  - Activated at the beginning of simulation
  - Executed once

- ***always***
  - Activated at the beginning of simulation
  - Repeats continuously throughout the whole simulation run

Initial

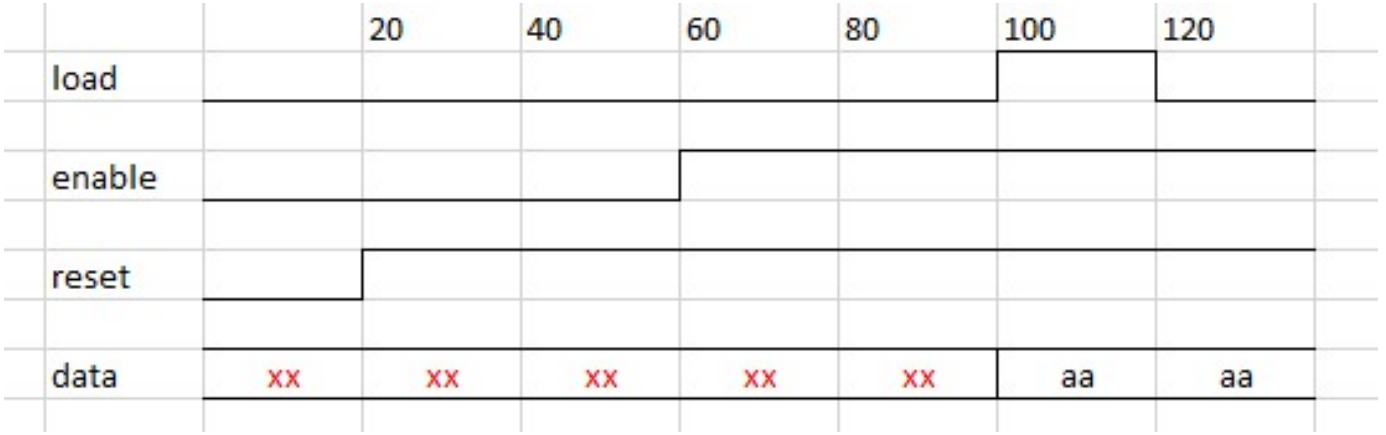
- Contains a statement or block of statements which is executed only once, starting at the beginning of simulation.
- An initial with more than one statement needs a begin-end block to enclose the statements.

```
1   reg Clock;
2   parameter HalfPeriod = 5;
3   initial
4   begin
5       Clock = 0;
6       #(HalfPeriod) Clock = ~Clock;
7   end
```



Initial Example

```
 1   reg Clock, Enable, Load, Reset;
 2   reg [7:0] Data;
 3   initial begin
 4       Load = 0;
 5       Enable = 0;
 6       Reset = 0;
 7       #20 Reset = 1;
 8       #40 Enable = 1;
 9       #40 Data = 8'haa;
10       Load = 1;
11       #20 Load = 0;
12   end
```



Always

- Only registers (reg, integer, real, time) may be assigned in an ***always*****.**
- Every **always starts executing at the start of simulation, and continues** executing throughout simulation; when the last statement in the **always is** reached, execution continues from the top of the **always.**
- An **always containing more than one statement must enclose the statements** in a begin-end block.
- An **always with no timing controls will loop forever.**